Vu en cours

- Données - Base de Données - SGBD
- Base de Données relationnelle
- Relation/table – Attribut/colonne – Tuple/ligne
- Clé primaire (simple ou composée)
- Clé étrangère (contrainte de référence)
- Algèbre relationnelle : projection et restriction

La base Employés

Base de données qui nous sert d'exemple: **Employés**

emp

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7839	KING	PRESIDENT		09-AUG-81	5000		10
7698	BLAKE	MANAGER	7839	21-JAN-81	2850		30
7782	CLARK	MANAGER	7839	01-MAR-81	2450		10
7566	JONES	MANAGER	7839	23-DEC-80	2975		20
.....							
.....							

dept

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

Clé primaire

Clé étrangère

Contrainte de référence

La base Employés

Base de données qui nous sert d'exemple : **Employés**

La table **emp**

empno	ename	job	mgr	hiredate	sal	comm	deptno

empno - numéro de l'employé, clé primaire

ename - nom

job - fonction

mgr – responsable hiérarchique, clé étrangère (contrainte de référence vers empno de emp)

hiredate - date d'embauche

sal - salaire

comm - commission (seulement pour les commerciaux, NULL pour les autres employés)

deptno - numéro du département dans lequel l'employé travaille, clé étrangère (contrainte de référence vers deptno de dept)

La base Employés

Base de données qui nous sert d'exemple: **Employés**

La table **dept**

deptno	dname	loc

deptno - numéro du département, clé primaire
dname - nom
loc - localisation

SQL (Structured Query Language)

- Une implantation des langages d'interrogation relationnels + d'autres commandes de gestion d'une BD relationnelle
- Issu de l'algèbre relationnelle et du calcul relationnel de tuples
- Retenu comme standard par l'ANSI en 1986
- L'algèbre relationnelle est pour SQL ce que l'algorithmique est pour la programmation : sa compréhension est indispensable pour écrire des requêtes

Bases du langage SQL

Fonctionnalités du SQL:

- **interrogation** (consultation de données)
- **mises à jour** (actualisation des données)
- **définition** (description) de la base de données
- **contrôle** - du traitement des données
  - de l'accès aux données

Interrogation de bases de données

- Rechercher les informations stockées dans la base répondant à une question sur les données
(= répondant à des critères donnés)
- Les langages d'interrogation relationnels permettent de décrire ces questions
- Une question est appelée **requête**

Exemple de question :

Quels sont les noms des analystes ?

→ La réponse est un ensemble de données

Langages d'interrogation relationnels

- L'algèbre relationnelle
  - Une requête spécifie un enchaînement d'opérations conduisant au résultat désiré (*j'applique à la relation « employé » une projection sur la colonne « nom » puis...*) -> procédural
    Le résultat d'une requête en algèbre relationnelle est une relation (un ensemble), chaque opération manipule une ou plusieurs relations et donne une relation.
- Les calculs relationnels (de tuples et de domaines)

Equivalence des langages

On peut démontrer que l'algèbre relationnelle, les calculs relationnels de tuples et les calculs des domaines sont équivalents : ils permettent de définir les mêmes ensembles de données.

Objet des TD/TP

La manipulation de bases de données

- TD : algèbre relationnelle
  principes et syntaxe SQL – Requêtes
- TP : mise en œuvre de SQL sur le SGBD Oracle

SQL - Requêtes simples

Table

Table = ensemble de lignes de même structure (mêmes colonnes, ou champs)

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7839	KING	PRESIDENT		09-AUG-81	5000		10
7698	BLAKE	MANAGER	7839	21-JAN-81	2850		30
7782	CLARK	MANAGER	7839	01-MAR-81	2450		10
7566	JONES	MANAGER	7839	23-DEC-80	2975		20
7654	MARTIN	SALESMAN	7698	20-JUN-81	1250	1400	30
7499	ALLEN	SALESMAN	7698	12-NOV-80	1600	300	30
7844	TURNER	SALESMAN	7698	31-MAY-81	1500	0	30
7900	JAMES	CLERK	7698	25-AUG-81	950		30
7521	WARD	SALESMAN	7698	14-NOV-80	1250	500	30
7902	FORD	ANALYST	7566	25-AUG-81	3000		20
7369	SMITH	CLERK	7902	08-SEP-80	800		20
7788	SCOTT	ANALYST	7566	31-AUG-82	3000		20
7876	ADAMS	CLERK	7788	04-OCT-82	1100		20
7934	MILLER	CLERK	7782	15-OCT-81	1300		10

Bases du langage SQL

Fonctionnalités du SQL:

- **interrogation : SELECT**
- **mises à jour : INSERT, UPDATE, DELETE**
- **définition : CREATE, ALTER, DROP**
- **contrôle : COMMIT, ROLLBACK**
  GRANT, REVOKE

Exemple de requête

Numéro et nom des employés qui ont un salaire supérieur à 2500.

```
SQL> SELECT empno, ename
  2  FROM emp
  3  WHERE sal>2500;

     EMPNO ENAME
---------- --------------------
      9999 GAG
      7839 KING
      7698 BLAKE
      7566 JONES
      7902 FORD
      7788 SCOTT

6 lignes selectionnees.

SQL>
```

SELECT : syntaxe générale

Requête d'interrogation = Commande SELECT

```
SELECT [ ALL | DISTINCT ]
        {  [ objet. ] * |  expression  [ c_alias ] }
          [ ,  {objet.* | expression  [ c_alias ]  } ] ...
FROM [ user. ] objet  [ t_alias ]  [ , [ user.] objet [ t_alias]  ] ...
[ WHERE Condition ]
[ GROUP BY expression [ , expression ] ...  ]
[ HAVING ConditionGroupe ]
[ { UNION  | INTERSECT  | MINUS } Requete ]
[ ORDER BY { expression |  position } [ ASC | DESC ]
                [ , { expression | position } [ ASC | DESC]  ]...]
[ FOR UPDATE OF colonne [ , colonne  ] ... ]  ;
```

Requêtes simples

Requête simple = une partie de la syntaxe de la commande SELECT

```
SELECT [ ALL | DISTINCT ]
       {  [ objet. ] * |  expression  [ c_alias ] }
          [ , {objet.* | expression  [ c_alias ] } ] ...
FROM [ user. ] objet  [ t_alias ]
[ WHERE Condition ]
[ ORDER BY { expression |  position } [ ASC | DESC ]
                    [ , { expression | position } [ ASC | DESC]  ]...]  ;
```

Notations
... : répétition possible de l'argument
[option] : argument facultatif
[option1 | option2 | ...] : argument facultatif, pouvant prendre les valeurs option1 ou option2... ; l'option soulignée est prise par défaut
{ option1 | option2 | ... } : obligatoirement l'une des options indiquées

Ces notations ne doivent pas apparaître dans la requête

Requêtes simples

Requête simple = une partie de la syntaxe de la commande SELECT

```
SELECT [ ALL | DISTINCT ]
{   [ objet. ] * |  expression   [ c_alias  ] }
          [ ,  {objet.* | expression  [ c_alias  ]  } ] ...

FROM [ user. ] objet  [ t_alias ]

[ WHERE Condition ]

[ ORDER BY { expression |  position } [ ASC | DESC ]
                  [ , { expression | position } [ ASC | DESC]  ]...]  ;
```

les informations retournées

la(les) table(s) source(s)

les conditions que doivent vérifier les lignes du résultat

les colonnes sur lesquelle(s) le résultat est trié

Requêtes simples : la plus simple

SELECT [objet.] * FROM objet [t_alias] ;

SELECT * FROM emp ; **(la table complète des employés)**

SELECT [objet.] * FROM objet [t_alias] ;

En algèbre relationnelle : r = emp

emp

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COM	DEPTNO
7839	KING	PRESIDENT		09-AUG-81	5000		10
7698	BLAKE	MANAGER	7839	21-JAN-81	2850		30
7782	CLARK	MANAGER	7839	01-MAR-81	2450		10
7566	JONES	MANAGER	7839	23-DEC-80	2975		20
...	...	...	...	...	...	...	...

Requêtes simples : la plus simple

SELECT [objet.] * FROM objet [t_alias] ;

SELECT [objet.] * FROM objet [t_alias] ;

SELECT * FROM emp ;

SELECT [objet.] * FROM objet [t_alias] ;

SELECT emp.* FROM emp ;

SELECT [objet.] * FROM objet [t_alias] ;

SELECT e.* FROM emp e ;

Requêtes simples : la plus simple

Toutes les informations sur (tous) les employés ?

SELECT * FROM emp ;

Réponse :

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
9999	GAG				3000		10
7839	KING	PRESIDENT		09-AUG-81	5000		10
7698	BLAKE	PLEADER	7839	21-JAN-81	2850		30
7782	CLARK	PLEADER	7839	01-MAR-81	2450		10
7566	JONES	PLEADER	7839	23-DEC-80	2975		20
7654	MARTIN	SALESMAN	7698	20-JUN-81	1250	1400	30
7499	ALLEN	SALESMAN	7698	12-NOV-80	1600	300	30
7844	TURNER	SALESMAN	7698	31-MAY-81	1500	0	30
7900	JAMES	CLERK	7698	25-AUG-81	950		30
7521	WARD	SALESMAN	7698	14-NOV-80	1250	500	30
7902	FORD	ANALYST	7566	25-AUG-81	3000		20
7369	SMITH	CLERK	7902	08-SEP-80	800		20
7788	SCOTT	ANALYST	7566	31-AUG-82	3000		20
7876	ADAMS	CLERK	7788	04-OCT-82	1100		20
7934	MILLER	CLERK	7782	15-OCT-81	1300		10

Requêtes simples : la plus simple

Toutes les informations sur (tous) les employés ?

SELECT * FROM emp ;

Toutes les informations sur les départements ?

……..

emp

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COM	DEPTNO
7839	KING	PRESIDENT		09-AUG-81	5000		10
7698	BLAKE	MANAGER	7839	21-JAN-81	2850		30
7782	CLARK	MANAGER	7839	01-MAR-81	2450		10
7566	JONES	MANAGER	7839	23-DEC-80	2975		20
...	...	...	...	...	...	...	...

dept

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

Requêtes simples : projection

Opérateur **proj** de l'algèbre relationnelle

r = proj(relation, {att1, att2, ...})

EN SQL : clause **SELECT**

```
SELECT [ ALL | DISTINCT ]
  {  [ objet. ] * |  expression  [ c_alias ]  }
       [ , {objet.* | expression  [ c_alias ]  } ] ...
FROM [ user. ] objet  [ t_alias ] ;
```

Requêtes simples : projection

```
SELECT [ ALL | DISTINCT ]
      {  [ objet. ] * |  expression  [ c_alias ] }
       [ , {objet.* | expression [ c_alias ] } ] ...
FROM [ user. ] objet  [ t_alias ] ;
```

Quels sont les noms et les salaires des employés ?

AR : proj(emp, {ename, sal})

SQL : ***SELECT ename, sal FROM emp ;***

```
SELECT {  [ objet. ] * |  expression  [ c_alias ] }
                  [ , {objet.* | expression [ c_alias ] } ] ...
                        FROM [ user. ] objet  [ t_alias ] ;
```

Requêtes simples : projection

SELECT [ALL | DISTINCT]
{ [objet.] * | expression [c_alias] }
[, {objet.* | expression [c_alias] }] ...
FROM [user.] objet [t_alias] ;

Quels sont les noms et les salaires des employés ?

AR : proj(emp, {ename, sal})

SQL : ***SELECT ename, sal FROM emp ;***
ou ***SELECT emp.ename, emp.sal FROM emp ;***
SELECT e.ename, e.sal FROM emp e ;
SELECT ename "Nom" , sal "Salaire" FROM emp ;

{ [objet.] * | expression [c_alias] }

Requêtes simples : projection

Quels sont les noms et les salaires des employés ?

AR : proj(emp, {ename, sal})

SQL : ***SELECT ename, sal FROM emp ;***

Réponse :

ENAME	SAL
GAG	3000
KING	5000
BLAKE	2850
CLARK	2450
JONES	2975
MARTIN	1250
ALLEN	1600
TURNER	1500
JAMES	950
WARD	1250
FORD	3000
SMITH	800
SCOTT	3000
ADAMS	1100
MILLER	1300

Requêtes simples : projection

Quels sont les noms et les salaires des employés ?

```
SELECT ename "Nom" , sal "Salaire"
FROM emp ;
```

Réponse :

Nom	Salaire
GAG	3000
KING	5000
BLAKE	2850
CLARK	2450
JONES	2975
MARTIN	1250
ALLEN	1600
TURNER	1500
JAMES	950
WARD	1250
FORD	3000
SMITH	800
SCOTT	3000
ADAMS	1100
MILLER	1300

Requêtes simples : projection

Quels sont les noms et les salaires des employés ?

```
SELECT ename "NOM" , sal "SALAIRE"
FROM emp ;
```

Réponse :

NOM	SALAIRE
GAG	3000
KING	5000
BLAKE	2850
CLARK	2450
JONES	2975
MARTIN	1250
ALLEN	1600
TURNER	1500
JAMES	950
WARD	1250
FORD	3000
SMITH	800
SCOTT	3000
ADAMS	1100
MILLER	1300

Requêtes simples : projection

Quels sont les fonctions des employés ?

SQL : ***SELECT job FROM emp ;***

Réponse :

JOB
PRESIDENT
PLEADER
PLEADER
PLEADER
SALESMAN
SALESMAN
SALESMAN
CLERK
SALESMAN
ANALYST
CLERK
ANALYST
CLERK
CLERK

Ne peut pas être obtenu en AR car pas de doublon

AR : proj(emp, {job}) (pas de doublon en AR)

SQL : ***SELECT DISTINCT job FROM emp;***

Réponse :

JOB
PRESIDENT
PLEADER
SALESMAN
CLERK
ANALYST

Requêtes simples : projection

Quels sont les noms et les fonctions des employés ?

SELECT ename, job FROM emp ;

Réponse :

ENAME	JOB
GAG	
KING	PRESIDENT
BLAKE	PLEADER
CLARK	PLEADER
JONES	PLEADER
MARTIN	SALESMAN
ALLEN	SALESMAN
TURNER	SALESMAN
JAMES	CLERK
WARD	SALESMAN
FORD	ANALYST
SMITH	CLERK
SCOTT	ANALYST
ADAMS	CLERK
MILLER	CLERK

SELECT DISTINCT ename, job FROM emp ;

DISTINCT porte sur toute la ligne

Requêtes simples : projection

Quels sont les noms et les salaires des employés ?

AR : proj(emp, {ename, sal})

SQL : ***SELECT ename, sal FROM emp ;***

Quels sont les fonctions (jobs) et les salaires des employés ?

……

Noms des employés et identifiant de leur chef ?

……

emp

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COM	DEPTNO
7839	KING	PRESIDENT		09-AUG-81	5000		10
7698	BLAKE	MANAGER	7839	21-JAN-81	2850		30
7782	CLARK	MANAGER	7839	01-MAR-81	2450		10
7566	JONES	MANAGER	7839	23-DEC-80	2975		20
…	…	…	…	…	…	…	…

dept

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

AR : la restriction (ou sélection, filtrage)

RelationRésultat = restrict (*relation*, *Condition*)

RelationRésultat :
- même schéma que relation
- se compose des n-uplets de *relation* qui vérifient *Condition*

Condition simple : *argument opérateur argument*
avec: - *opérateur* = opérateur de comparaison (=, >, ...)
- *argument* = nom d'attribut
expression calculée
constante

Une condition simple peut être combinée avec d'autres conditions à l'aide des opérateurs et ($\wedge$), ou($\vee$).

Requêtes simples : restriction

En SQL : clause **WHERE**

Informations sur les employés du département 10 ?

AR : ?

SQL : ***SELECT ****
FROM emp
WHERE deptno = 10 ;

AR : restriction - Exemples

Informations sur les employés dont la fonction est « analyst » ?

AR : ……

SQL : ……

Informations sur les employés de salaire supérieur à 2000 ?

AR : ……

SQL : ……

emp

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COM	DEPTNO
7839	KING	PRESIDENT		09-AUG-81	5000		10
7698	BLAKE	MANAGER	7839	21-JAN-81	2850		30
7782	CLARK	MANAGER	7839	01-MAR-81	2450		10
7566	JONES	MANAGER	7839	23-DEC-80	2975		20
...	...	...	...	...	...	...	...

dept

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

Requêtes simples : projection+restriction

Une requête d'AR est une séquence d'opérations à appliquer pour arriver au résultat souhaité.

L'application d'un opérateur d'AR sur des relations donne une relation.

Quels sont les noms et les salaires des employés du département 10 ?

AR : r1 = restrict(emp, deptno=10)
res = proj(r1, {ename, sal})

ou res = proj(restrict(emp, deptno=10), {ename, sal})

Requêtes simples : conditions de sélection

argument1 opérateur argument2

opérateur (de comparaison): = != <> < <= > >=
argument1 , ***argument2*** :
- constantes
- noms de colonnes
- expressions avec des opérateurs (+, -, /, *)

et/ou des fonctions appliquées à une (ou plusieurs) colonne(s)
(voir plus *loin Expressions & fonction*)

Informations sur les employés dont le salaire est supérieur à 1000 ?

SQL : ***SELECT * FROM emp WHERE sal > 1000 ;***

Informations sur les analystes ?

SQL : ***SELECT * FROM emp WHERE job = 'ANALYST' ;***

Quels sont les noms et les salaires des employés du département 10 ?

SQL : ***SELECT ename, sal FROM emp WHERE deptno = 10 ;***

Requêtes simples : projection+restriction

Quels sont les numéros et dates d'embauche des employés du département 20 ?

AR : ……

SQL : ……

emp

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COM	DEPTNO
7839	KING	PRESIDENT		09-AUG-81	5000		10
7698	BLAKE	MANAGER	7839	21-JAN-81	2850		30
7782	CLARK	MANAGER	7839	01-MAR-81	2450		10
7566	JONES	MANAGER	7839	23-DEC-80	2975		20
...	...	...	...	...	...	...	...

dept

DEPTNO	DNAME	LOC
10	ACCOUNTING	NEW YORK
20	RESEARCH	DALLAS
30	SALES	CHICAGO
40	OPERATIONS	BOSTON

Vérifier une requête

Le résultat de la requête doit au moins correspondre à ce que l'on attend compte tenu de ce qu'il y a dans la table (attention ce n'est pas une preuve absolue car il serait possible que la requête fonctionne sur les données existantes, mais ne fonctionne plus s'il y avait un petit changement)

Résultat attendu ?

Quels sont les numéros et dates d'embauche des employés du département 20 ?

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7839	KING	PRESIDENT		09-AUG-81	5000		10
7698	BLAKE	MANAGER	7839	21-JAN-81	2850		30
7782	CLARK	MANAGER	7839	01-MAR-81	2450		10
7566	JONES	MANAGER	7839	23-DEC-80	2975		20
7654	MARTIN	SALESMAN	7698	20-JUN-81	1250	1400	30
7499	ALLEN	SALESMAN	7698	12-NOV-80	1600	300	30
7844	TURNER	SALESMAN	7698	31-MAY-81	1500	0	30
7900	JAMES	CLERK	7698	25-AUG-81	950		30
7521	WARD	SALESMAN	7698	14-NOV-80	1250	500	30
7902	FORD	ANALYST	7566	25-AUG-81	3000		20
7369	SMITH	CLERK	7902	08-SEP-80	800		20
7788	SCOTT	ANALYST	7566	31-AUG-82	3000		20
7876	ADAMS	CLERK	7788	04-OCT-82	1100		20
7934	MILLER	CLERK	7782	15-OCT-81	1300		10

Résultat attendu ?

Quels sont les numéros et dates d'embauche des employés du département 20 ?

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7839	KING	PRESIDENT		09-AUG-81	5000		10
7698	BLAKE	MANAGER	7839	21-JAN-81	2850		30
7782	CLARK	MANAGER	7839	01-MAR-81	2450		10
7566	JONES	MANAGER	7839	23-DEC-80	2975		20
7654	MARTIN	SALESMAN	7698	20-JUN-81	1250	1400	30
7499	ALLEN	SALESMAN	7698	12-NOV-80	1600	300	30
7844	TURNER	SALESMAN	7698	31-MAY-81	1500	0	30
7900	JAMES	CLERK	7698	25-AUG-81	950		30
7521	WARD	SALESMAN	7698	14-NOV-80	1250	500	30
7902	FORD	ANALYST	7566	25-AUG-81	3000		20
7369	SMITH	CLERK	7902	08-SEP-80	800		20
7788	SCOTT	ANALYST	7566	31-AUG-82	3000		20
7876	ADAMS	CLERK	7788	04-OCT-82	1100		20
7934	MILLER	CLERK	7782	15-OCT-81	1300		10

Résultat attendu ?

Quels sont les numéros et dates d'embauche des employés du département 20 ?

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
7839	KING	PRESIDENT		09-AUG-81	5000		10
7698	BLAKE	MANAGER	7839	21-JAN-81	2850		30
7782	CLARK	MANAGER	7839	01-MAR-81	2450		10
7566	JONES	MANAGER	7839	23-DEC-80	2975		20
7654	MARTIN	SALESMAN	7698	20-JUN-81	1250	1400	30
7499	ALLEN	SALESMAN	7698	12-NOV-80	1600	300	30
7844	TURNER	SALESMAN	7698	31-MAY-81	1500	0	30
7900	JAMES	CLERK	7698	25-AUG-81	950		30
7521	WARD	SALESMAN	7698	14-NOV-80	1250	500	30
7902	FORD	ANALYST	7566	25-AUG-81	3000		20
7369	SMITH	CLERK	7902	08-SEP-80	800		20
7788	SCOTT	ANALYST	7566	31-AUG-82	3000		20
7876	ADAMS	CLERK	7788	04-OCT-82	1100		20
7934	MILLER	CLERK	7782	15-OCT-81	1300		10

restriction projection

Résultat attendu ?

Quels sont les numéros et dates d'embauche des employés du département 20 ?

EMPNO	HIREDATE
7566	23-DEC-80
7902	25-AUG-81
7369	08-SEP-80
7788	31-AUG-82
7876	04-OCT-82

ATTENTION

- ORACLE est sensible à la casse pour les chaînes de caractères !!!

'ANALYST' ≠ 'Analyst'

- Les chaînes de caractères sont entre '

Déroulement de l'exécution d'une requête

empno	*ename*	*job*	*mgr*	*hiredate*	*sal*	*comm*	*deptno*
100	DUPONT	ANALYST	500	08-SEP-80	1900		20
200	DURAND	PLEADER		20-JUN-81	2300		10
300	DUPONT	ANALYST	500	08-SEP-80	1900		20
400	MARTIN	SALESMAN	200	14-NOV-80	1975	200	10

```
select *
from emp
where deptno=20
```

Résultat
?

Déroulement de l'exécution d'une requête

A chaque instant, seules les données de la ligne courante sont accessibles

empno	ename	job	mgr	hiredate	sal	comm	deptno
100	DUPONT	ANALYST	500	08-SEP-80	1900		20
200	DURAND	PLEADER		20-JUN-81	2300		10
300	DUPONT	ANALYST	500	08-SEP-80	1900		20
400	MARTIN	SALESMAN	200	14-NOV-80	1975	200	10

```
select *
from emp
where deptno=20
```

Résultat
?

Déroulement de l'exécution d'une requête

empno	ename	job	mgr	hiredate	sal	comm	deptno
100	DUPONT	ANALYST	500	08-SEP-80	1900		20
200	DURAND	PLEADER		20-JUN-81	2300		10
300	DUPONT	ANALYST	500	08-SEP-80	1900		20
400	MARTIN	SALESMAN	200	14-NOV-80	1975	200	10





```
select *
from emp
where deptno=20
```

Résultat
?

Les lignes sont scrutées les unes après les autres, pour chacune la condition spécifiée par la clause WHERE est vérifiée

Déroulement de l'exécution d'une requête

empno	ename	job	mgr	hiredate	sal	comm	deptno
100	DUPONT	ANALYST	500	08-SEP-80	1900		20
200	DURAND	PLEADER		20-JUN-81	2300		10
300	DUPONT	ANALYST	500	08-SEP-80	1900		20
400	MARTIN	SALESMAN	200	14-NOV-80	1975	200	10





```
select *
from emp
where deptno=20
```

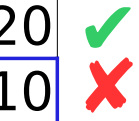
Lorsque la condition est vérifiée, la ligne est conservée

Résultat

```
EMPNO ENAME   JOB         MGR HIREDATE SAL  COMM  DEPTNO
----- ------- --------- ----- -------- ---- ----- ------
  100 DUPONT  ANALYST     500 08/09/80 1900            20
```

Déroulement de l'exécution d'une requête

empno	ename	job	mgr	hiredate	sal	comm	deptno
100	DUPONT	ANALYST	500	08-SEP-80	1900		20
200	DURAND	PLEADER		20-JUN-81	2300		10
300	DUPONT	ANALYST	500	08-SEP-80	1900		20
400	MARTIN	SALESMAN	200	14-NOV-80	1975	200	10



```
select *
from emp
where deptno=20
```

Lorsque la condition n'est pas vérifiée, la ligne est n'est pas prise

Résultat

```
EMPNO ENAME   JOB         MGR HIREDATE SAL  COMM  DEPTNO
----- ------- --------- ----- -------- ---- ----- ------
  100 DUPONT  ANALYST     500 08/09/80 1900             20
```

Déroulement de l'exécution d'une requête

empno	*ename*	*job*	*mgr*	*hiredate*	*sal*	*comm*	*deptno*	
100	DUPONT	ANALYST	500	08-SEP-80	1900		20	✔
200	DURAND	PLEADER		20-JUN-81	2300		10	✘
300	DUPONT	ANALYST	500	08-SEP-80	1900		20	✔
400	MARTIN	SALESMAN	200	14-NOV-80	1975	200	10	

```
select *
from emp
where deptno=20
```

Résultat

```
EMPNO ENAME   JOB          MGR HIREDATE SAL  COMM  DEPTNO
----- ------- --------- ----- -------- ---- ----- ------
  100 DUPONT  ANALYST     500 08/09/80 1900             20
  300 DUPONT  ANALYST     500 08/09/80 1900             20
```

Déroulement de l'exécution d'une requête

empno	ename	job	mgr	hiredate	sal	comm	deptno	
100	DUPONT	ANALYST	500	08-SEP-80	1900		20	✔
200	DURAND	PLEADER		20-JUN-81	2300		10	✘
300	DUPONT	ANALYST	500	08-SEP-80	1900		20	✔
→ 400	MARTIN	SALESMAN	200	14-NOV-80	1975	200	10	✘

```
select *
from emp
where deptno=20
```

Une fois toutes les lignes scrutées, la projection spécifiée par la clause SELECT est réalisée (aucune sur cet exemple)

Résultat

```
EMPNO ENAME   JOB          MGR HIREDATE SAL  COMM  DEPTNO
----- ------- --------- ----- -------- ---- ----- ------
  100 DUPONT  ANALYST      500 08/09/80 1900             20
  300 DUPONT  ANALYST      500 08/09/80 1900             20
```

Requêtes simples : conditions de sélection

colonne IS [NOT] NULL

Informations sur les employés qui perçoivent une commission ? (les commerciaux)
SELECT * FROM emp WHERE comm IS NOT NULL ;

Informations sur les employés qui ne perçoivent pas de commission ?
SELECT * FROM emp WHERE comm IS NULL ; (les non commerciaux)

Informations sur les employés qui perçoivent 0 de commission ?
SELECT * FROM emp WHERE comm = 0 ; (les mauvais commerciaux)

Requêtes simples : conditions de sélection

argument **[NOT] IN (*liste*)**
où *liste* est une suite de valeurs séparées par le symbole ,

IN et =

EMPNO	ENAME	JOB	MGR	HIREDATE	SAL	COMM	DEPTNO
100	DUPONT	ANALYST	500	08/09/80	1900		20

ENAME IN ('DUPONT', 'DUPOND', 'MARTIN', 'MERLE') vrai
ENAME IN ('DUPOND', 'MARTIN', 'MERLE') faux !!
ENAME NOT IN ('MARTIN', 'MERLE') vrai

ENAME = 'DUPONT' vrai
ENAME = 'Dupont' faux !!
ENAME <> 'Dupont' vrai

Requêtes simples : conditions de sélection

argument **[NOT] IN (SELECT ...)**

(*argument*, *argument*, ...) [NOT] IN (SELECT ...)

(vu plus tard)

Requêtes simples : conditions de sélection

argument **[NOT] BETWEEN valeur AND valeur**

Informations sur les employés dont le salaire est compris entre 800 et 2000 ?
SELECT * FROM emp WHERE sal BETWEEN 800 and 2000 ;

Informations sur les employés dont le salaire n'est pas compris entre 800 et 2000 ?
SELECT * FROM emp WHERE sal NOT BETWEEN 800 and 2000 ;

Requêtes simples : conditions de sélection

argument **[NOT] LIKE 'chaîne générique'**

Chaîne générique

Motif qui donne la forme d'une chaîne
Exemples :

'S%' : chaînes commençant par S (S...)
Instances : Strasbourg, Sidney...

'%s' : chaînes se terminant par s (...s)
Instances : Paris, Rennes...

'_a%' : chaînes comportant un 'a' en 2e position
Instances : Paris, Nantes...

'_a_ _ _' : chaînes de 5 lettres avec un a en 2e position
Par i s
Lava l

LIKE

'Comme'

'Paris' LIKE 'P%' vrai
'S%' faux !!
'_a%' vrai
'_a_ _ _' vrai
...

'Paris' NOT LIKE 'S%' vrai
'P%' faux !!
'_ _ _ r %' vrai
'_ A_ _ _' (sous oracle)
...

LIKE et =

LIKE : « c'est de la forme... »

= : « c'est exactement »

'Paris' LIKE 'P%' vrai

'Paris' = 'P%' faux !!

'Paris' != 'P%' vrai

Conditions

Expressions booléennes : valent vrai ou faux

OR, AND et NOT s'appliquent à des conditions

cond1 OR cond2 => vrai si ?

cond1 AND cond2 => vrai si ?

not(cond) => vrai si ?

Conditions

```
EMPNO      ENAME      JOB        MGR    HIREDATE  SAL  COMM        DEPTNO
-----      -------    ---------  -----  --------  ----  -----      ------
  100      DUPONT     ANALYST    500    08/09/80  1900                 20
```

```
select * from emp where deptno = 20

select * from emp where deptno = 20 or deptno = 10

select * from emp where deptno = 20 and deptno = 10

select * from emp where job = 'ANALYST' and deptno = 20

select * from emp where deptno = 20 or 10 ???

select * from emp where deptno = 20 or job = 'ANALYST'
          and job = 'SALESMAN' ???
```

Requêtes simples : trier le résultat

```
SELECT [ ALL | DISTINCT ]
        {  [ objet. ] * |  expression  [ c_alias ] }
         [ ,  {objet.* | expression  [ c_alias ]  } ] ...
FROM [ user. ] objet  [ t_alias ]
[ ORDER BY { expression |  position } [ ASC | DESC ]
                [ , { expression | position } [ ASC | DESC]  ]...] ;
```

Informations sur les employés classées par No de département décroissant (critère principal) et salaire croissant (critère secondaire pour départager les employés du même département) ?

SELECT * FROM emp ORDER BY deptno DESC, sal ;

Expressions et fonctions arithmétiques

Expressions et fonctions

sont utilisables dans les clauses
SELECT, WHERE,

ORDER BY (et HAVING)

Les expressions

comprennent :

- **des noms de colonnes**
- **des opérateurs**
  - **arithmétiques : + - * /**
  - **pour les chaînes : || (concaténation)**
  - **pour les dates : + - (voir diapo suivante)**
  - **booléens : AND OR NOT**
- **des fonctions (présentées dans la suite)**

Expressions : exemples

Opérateurs arithmétiques

`4*5+4*6/3` → **28**

`(4*5)+(4*6)/3` → **28**

`4*(5+4)*6/3` → **72**

Bien parenthéser vos opérations pour éviter les problèmes

`(sal+comm)*12`

Expressions : exemples

Booléens

```
(2<3) AND (7=5) → False
(2<3) OR (7=5) → True
(2<3) OR (7=5) AND (1>4) → True
((2<3) OR (7=5)) AND (1>4) → False
'OMEGA' < 'ALPHA' → False
'Alpha' < 'ALPHA' → False
'OMEGA' < 'alpha' → True
'Alpha' < 'Alphonse' → True
```

Expressions

comprennent:

- **noms de colonnes**
- **opérateurs**
- **fonctions**

Fonctions

Elles différent selon le SGBD !

Fonctions arithmétiques (Oracle)

ABS (n)	valeur absolue
CEIL (n)	le plus petit entier qui dépasse n
FLOOR (n)	le plus grand entier inférieur à n
MOD (m,n)	reste de la division de m par n
POWER (m,n)	m est élevé à la puissance n
ROUND (n [,m])	n est arrondi à m décimales (m=0 si omis)
TRUNC (n [,m])	n est tronqué à m décimales (m=0 si omis)
SQRT (n)	racine carrée
LEAST (n1,n2,...)	la plus faible valeur parmi n1,n2, ...
GREATEST (n1,n2,...)	la plus forte valeur parmi n1,n2, ...

Fonctions arithmétiques : exemples

```
select ename, greatest(sal,comm) from emp

select empno, ename, least(round(sal), round(comm))
from emp
where round(sal)>1500
```

Ce que sont les requêtes simples

Des requêtes permettant d'obtenir des données stockées dans une unique table, vérifiant des critères simples obtenus à partir des données de la table

Auteurs

Ce cours est issu du travail de

- Agnès Braud
- Julien Schnell
- Cornelia Tudorie